

Using Distributed Reinforcement Learning for Resource Orchestration in a Network Slicing Scenario

Federico Mason[✉], *Graduate Student Member, IEEE*, Gianfranco Nencioni[✉],
and Andrea Zanella[✉], *Senior Member, IEEE*

Abstract—The Network Slicing (NS) paradigm enables the partition of physical and virtual resources among multiple logical networks, possibly managed by different tenants. In such a scenario, network resources need to be dynamically allocated according to the slice requirements. In this paper, we attack the above problem by exploiting a Deep Reinforcement Learning approach. Our framework is based on a distributed architecture, where multiple agents cooperate towards a common goal. The agent training is carried out following the Advantage Actor Critic algorithm, which permits to handle continuous action spaces. By means of extensive simulations, we show that our approach yields better performance than both a static allocation of system resources and an efficient empirical strategy. At the same time, the proposed system ensures high adaptability to different scenarios without the need for additional training.

Index Terms—Network slicing, resource allocation, distributed machine learning, deep reinforcement learning.

I. INTRODUCTION

THE fifth generation of cellular networks (5G) aims at supporting different applications with very specific requirements over the same infrastructure. In this perspective, the 3GPP consortium has identified three main service classes, namely enhanced Mobile BroadBand (eMBB), Ultra Reliable Low Latency Communication (URLLC) and massive Machine Type Communication (mMTC) [1]. Specifically, eMBB is expected to provide very high throughput in both downlink and uplink, URLLC appeals to applications with strict latency and reliability constraints, while mMTC is intended to support thousands of monitoring or sensing devices that communicate with minimal energetic and computational costs [2].

Manuscript received 24 December 2020; revised 31 May 2021, 29 January 2022, and 11 May 2022; accepted 18 June 2022; approved by IEEE/ACM TRANSACTIONS ON NETWORKING Editor K. Lee. Date of publication 1 July 2022; date of current version 16 February 2023. This work was supported in part by the Consortium Gestione Ampliamento Rete Ricerca (GARR) through the “Orio Carlini” Scholarship 2019 and in part by the Norwegian Research Council through the 5G-MODaNeI Project under Grant 308909. A preliminary and reduced version of this manuscript has been published in *2021 19th Mediterranean Communication and Computer Networking Conference* [DOI: 10.1109/MedComNet52149.2021.9501279]. (Corresponding author: Federico Mason.)

Federico Mason and Andrea Zanella are with the Department of Information Engineering, University of Padova, 35131 Padova, Italy (e-mail: masonfed@dei.unipd.it; zanella@dei.unipd.it).

Gianfranco Nencioni is with the Department of Electrical Engineering and Computer Science, University of Stavanger, 4021 Stavanger, Norway (e-mail: gianfranco.nencioni@uis.no).

Digital Object Identifier 10.1109/TNET.2022.3187310

Traditional telecommunication networks are often based on a rigid architecture and are not apt to support such services [3]. To overcome this problem, the research community has introduced the concepts of Software Defined Networking (SDN) and Network Function Virtualization (NFV), which can make networks more flexible and adaptable to different requirements [4]. The NFV principle makes it possible to execute network functions over general-purpose devices or virtual machines in the Cloud. Instead, SDN separates the control plane from the forwarding plane, enabling dynamic and differentiated routing of data flows.

In particular, the SDN and NFV concepts are key enablers of the Network Slicing (NS) paradigm, which makes it possible to define multiple virtual networks over the same physical infrastructure [5], [6]. Under this vision, a slice consists of a virtual overlay network designed to support communication services with similar characteristics [7]. Hence, a slice supporting eMBB applications (e.g., video streaming) is characterized by very high bit rate, while a slice supporting URLLC applications (e.g., telesurgery) guarantees extremely high reliability and low latency.

If defined over the same infrastructure, different slices will contend for the same resources, which can be both physical (e.g., optical links) and virtual (e.g., virtual baseband processing units) [8]. In general, such resources are acquired by the *slice broker* (i.e., the body in charge of initializing and orchestrating slices) from the *infrastructure providers* (i.e., the owners of the physical elements of the network) [9]. Then, slices are assigned to the *slice tenants* (e.g., virtual network operators), which offer slice services to the end-users. The amount of resources required by a certain slice is specified in the so-called *Service Level Agreement (SLA) between the slice tenant and broker* [10]. Therefore, a fundamental challenge in NS systems is how to distribute resources among the different slices in an efficient way, ensuring that all the SLAs are satisfied [11].

In this work, we consider a scenario where two slice classes (i.e., eMBB and URLLC), with dynamic and complementary requirements in term of throughput, computational power, memory capacity, and delay, are instantiated over the same network infrastructure. Each slice is composed by multiple information flows (with static routes) that contend for the bandwidth provided by network links, and for the

computational and memory resources of the network nodes. The problem is to dynamically distribute the network resources among the active information flows, in accordance to the characteristic of the slices that they belong to.

The *naïve* approach consists in statically allocating communication and computational resources across the slices. However, this method cannot exploit the statistical multiplexing of the information flows and, consequently, may lead to greater over-provisioning costs and low utilization of the available resources. On the other hand, conventional allocation schemes are generally unsuitable for scenarios in which the type, quantity, and location of resources can be very disparate, and the utility functions are non-convex. These characteristics make the resource allocation in NS a very challenging problem, generally incompatible with a centralized approach.

In this paper, we propose a Machine Learning (ML) approach and attack the problem by exploiting the Deep Reinforcement Learning (DRL) paradigm, which combines Reinforcement Learning (RL) algorithms and Neural Networks (NNs) to find strategies for the management of complex environments [12]. More specifically, we design a learning system where multiple agents collaborate to allocate network resources among the different slices running over the same infrastructure. The continuous interaction between such learning units makes it possible to increase both the system efficiency and adaptability to different scenarios.

Our work's contributions consist in the following points:

- We introduce a general network model that represents multiple communication slices running over the same infrastructure in different configurations.
- We develop a novel resource allocation strategy, based on DRL, that makes it possible to address the requirements of multiple slices in virtually any network topology.
- We show how *transfer learning* can improve the performance of the system, by specializing the strategy learned by the agents in a scalable manner.

The performance of the proposed strategy is assessed in multiple scenarios, including a real network topology, and compared with a meta-heuristic technique and an efficient empirical algorithm to analyze the advantages and limits of our solution.

To our knowledge, our proposal is one of the first attempts to orchestrate network core resources through a fully distributed strategy, where each agent takes decisions with a partial vision of the overall network status. Besides, the proposed approach makes it possible to exploit transfer learning in order to specialize policies learned in certain network topologies to other scenarios, without the need to re-train everything from scratch. Hence, if the network topology or the slice characteristics change, the single agents can adapt to the new scenario through a limited number of training episodes.

The remainder of the work is organized as follows. Sec. II discusses the most relevant works in the considered field. Sec. III describes the system model used for our analysis. Sec. IV recalls the fundamentals of DRL and describes our learning architecture. Sec. V presents the resource allocation

strategies used a benchmark. Sec. VI describes the simulation scenario and presents the results of our research. Finally, Sec. VII concludes the paper with a recap of the lessons learned and some ideas for future work.

II. RELATED WORK

In the last years, the emergence of the NS paradigm has made it necessary to investigate new strategies to orchestrate telecommunication systems. The authors of [13] analyze a 5G scenario with end-to-end slices contending for virtual resources offered by data centers, and propose a fully distributed algorithm to maximize the slice utility functions, which are assumed to be strictly concave. In [14], Leconte *et al.* design a NS model where multiple traffic flows share network bandwidth and cloud processing units; hence, they implement the Alternating Direction Method of Multipliers [15] to determine the best resource allocation scheme. In [16], a similar approach is applied to a system where multiple network operators share both licensed and unlicensed spectrum. Besides, the authors of [17] focus on the problem of offloading user tasks to edge computing facilities and design a novel algorithm to optimize server utilization, even in the case task execution time is not known. Finally, Fossati *et al.* propose a framework to adapt multi-resource allocation to with different fairness goals, considering also the critical scenario where resources are not sufficient to satisfy all the slice demands [18].

Because of the great complexity of NS scenarios, many studies have also proposed ML-based solutions. For instance, the authors of [19] exploit NNs to predict the traffic evolution in a mobile core-network, thus optimizing the routing and the wavelength assignment according to the SDN principles. Another example can be found in [20], where generative adversarial NNs are used to minimize the noise in the measurement of SLA satisfaction. Instead, [21] proposes a system based on convolutional NNs to associate users with network slices according to the required Quality of Service (QoS). Finally, in [22], Saputra *et al.* implement a distributed architecture predicting the amount of data that has to be cached in the network edge to address the user demands.

Among all the ML techniques used for slice orchestration, DRL is particularly appreciated for its ability to learn complex strategies by trial and error, without the need for labeled data. In this regard, the authors of [23] propose a centralized DRL scheme to split the wireless bandwidth among different slices in a single-cell system, targeting the maximization of spectral efficiency and SLA compliance. A similar framework is presented in [24], where a DRL agent optimizes the throughput of users associated with different slices, each with its own QoS requirements. The proposed algorithm makes it possible to achieve the optimal resource allocation, provided that the number of resource blocks does not vary over time. Another example is given in [25], where the authors consider a more complex system with two optimization layers, each characterized by a different set of actions. At the higher layer, a DRL agent tunes the users' data rates, considering the slice traffic requirements over a long time horizon; instead, at the lower layer, a second agent allocates the cell bandwidth among the users over a short time interval.

TABLE I
MODEL PARAMETERS

Parameter	Description	Parameter	Description	Parameter	Description
$\phi \in \Phi$	Information flow	$\epsilon_\phi^i, \epsilon_\phi^e$	Flow endpoints	B_l	Link rate capacity [bps]
$\sigma \in \Sigma$	Slice class	$\mathbf{r}_\phi$	Flow demand vector	C_n^c	Node computational capacity [bps]
$l \in \mathcal{L}$	Link	ρ_ϕ	Resource required by ϕ	C_n^m	Node memory capacity [b]
$n \in \mathcal{N}$	Node	$\hat{\rho}_\phi$	Resource assigned to ϕ	$b_{l,\phi}^i$	Input flow rate [bps]
η	Throughput [bps]	$f_\sigma(\cdot)$	Resource performance function	$b_{l,\phi}^o$	Output flow rate [bps]
c	Computational power [bps]	$F_\sigma(\cdot)$	Flow performance function	τ_n	Node routing delay [s]
m	Memory [b]	Ω	System utility	$D_{l,\phi}$	Data of ϕ queued in l [b]
δ	Delay [s]	$b_{l,\phi}$	Bit rate assigned by l to ϕ [bps]	$\tau_{l,\phi}^q$	Queuing delay of ϕ in l [s]
t	Discrete time	$c_{n,\phi}$	Computation assigned by n to ϕ [bps]	$\tau_{l,\phi}^t$	Transmission delay of ϕ in l [s]
T	Timeslot duration [s]	$m_{n,\phi}$	Memory assigned by n to ϕ [b]	τ_l^p	Link propagation delay [s]

A further step towards the development of a comprehensive DRL framework for NS is given in [26], where the authors consider an environment with multiple slices contending for the computational and bandwidth resources provided by a set of nodes. In this case, a centralized DRL agent has a global view of the overall network status and, at each slot, varies the number of resources reserved by a node to a certain slice. The agent receives a reward proportional to the efficiency of the resource allocation and the SLA compliance. Finally, the work presented in [27] focuses on the cooperation between multiple base stations that belong to the same network. The authors consider a multi-agent scenario where each base station is associated to a DRL agent that determines its bandwidth allocation. In particular, the agents can estimate the overall network status by exchanging embedding vectors, which are computed via a Graph NN system.

Despite the growing interest in the exploitation of DRL for slice orchestration, many questions remain open to investigation. In the current literature, DRL is used to optimize only a specific portion of the network (e.g., the wireless access), neglecting the interactions among different learning units. In particular, most of the above approaches assume that the DRL agent has a fully vision on the target system, while in more complex (and realistic) scenarios, the network status can be only partially observed, or with some delay. Finally, to the best of our knowledge, there are no scientific works investigating the adaptability of DRL solutions to different network topologies, which is a key aspect of slice orchestration.

The great heterogeneity of future telecommunication systems requires the implementation of more flexible strategies, which enable the coexistence of multiple services and can promptly adapt to new resource demands. A very promising approach is to exploit hierarchical reinforcement learning, which has not yet been fully investigated in this context. Moreover, the transfer learning paradigm can be used to improve the training of the learning agents [28], thus increasing the system adaptability to multiple scenarios. Our work develops along these directions, with the final aim of designing a fully scalable DRL system that can be separated into smaller units, capable of both acting autonomously and cooperating to orchestrate network resources under multiple working conditions.

III. SYSTEM MODEL

In this section, we model an NS environment in which multiple information flows contend for the same physical and

virtual network resources. We adopt a fluid traffic model, where the traffic through a link is viewed as a fluid stream of data with a certain flow rate. In particular, we assume that network slices are organized hierarchically, so that more flows at different levels can be compound into an aggregate slice, possibly managed by different tenants. The resulting framework is thus very flexible, and can model the interactions among many of the actors involved in a NS scenario. For the reader convenience, we report the main parameters of our model in Tab. I.

A. Slice Model

In our system, we define a *network slice* as an aggregation of information flows with similar behaviors and requirements. We denote by Σ the set including all the different classes of slices, and by Φ the set including all the information flows. Given $\sigma \in \Sigma$, we indicate by Φ_σ the set of all information flows of class σ . Each information flow $\phi \in \Phi$ is characterized by a tuple of parameters, namely:

- the *flow endpoints* $\mathcal{E}_\phi = (\epsilon_\phi^i, \epsilon_\phi^e)$, which correspond to the network nodes where the users' data enter/exit the slice and usually correspond to base stations, edge routers of autonomous systems, or servers;
- the *resource demand vector* $\mathbf{r}_\phi = [\eta_\phi, c_\phi, m_\phi, \delta_\phi]$, whose elements are the requirements in terms of throughput (η), computational power (c), memory (m), and maximum delay (δ) of the flow;
- the *performance function* $F_\phi(\cdot)$, which describes the performance of the information flow for a given level of fulfillment of its SLA.

We assume that $\mathcal{E}_\phi$, δ_ϕ and $F_\phi(\cdot)$ do not change for the whole duration of the flow ϕ , while η_ϕ , c_ϕ and m_ϕ may change in time, depending on the dynamic of the data source. Note that the throughput η_ϕ is measured in bits per second [b/s], the memory m_ϕ in bits [b], and the delay δ_ϕ in seconds [s]. Finally, we assume the computational requirements are somehow related to the amount of data generated by the flow source. We hence define the computational power c_ϕ as the speed at which data are processed in the computing facilities, and expressed it in bits per second [bps].

In our framework, the time is discretized in timeslots of T seconds, and we assume that the information flow parameters can change only slot by slot. We write $\mathbf{r}_\phi = [\eta_\phi(t), c_\phi(t), m_\phi(t), \delta_\phi(t)]$ and $\hat{\mathbf{r}}_\phi(t) = [\hat{\eta}_\phi(t), \hat{c}_\phi(t), \hat{m}_\phi(t), \hat{\delta}_\phi(t)]$ to

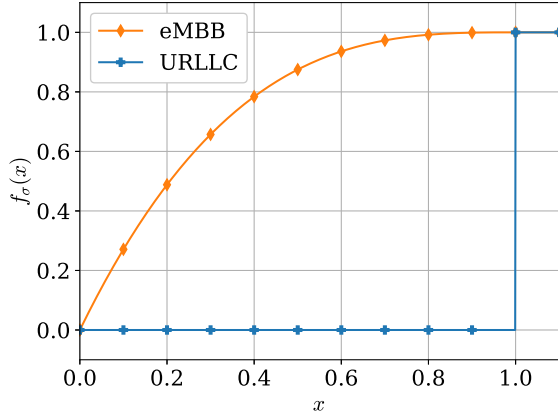


Fig. 1. Resource performance function.

indicate the resource vector demanded by and allocated to ϕ during timeslot t , respectively. Note that $\mathbf{r}_\phi(t)$ is determined by the slice class σ that ϕ belongs to, while $\hat{\mathbf{r}}_\phi(t)$ is determined by the resource allocation strategy.

As mentioned, we consider two different slice classes: eMBB (e) and URLLC (u). In particular, given Φ_e (i.e., the set of the eMBB flows) and Φ_u (i.e., the set of the URLLC flows), we have $\Phi = \Phi_e \cup \Phi_u$ and $\Phi_e \cap \Phi_u = \emptyset$. All the information flows of class σ shares the same performance function $F_\sigma(\cdot)$, i.e., $\forall \phi \in \Phi_\sigma, F_\phi(\cdot) = F_\sigma(\cdot)$. In general, $F_\sigma(\cdot)$ depends on both $\mathbf{r}$ and $\hat{\mathbf{r}}$, and returns a value in $[0, 1]$, where 1 means that the SLA has been completely fulfilled. We assume that $F_\sigma(\cdot)$ is a combination of four identical functions $f_\sigma(x_\rho) \in [0, 1]$, with $\rho \in \{\delta, \eta, c, m\}$, where $x_\rho \in [0, 1]$ indicates the level at which the demand for resource ρ is satisfied. The function $f_\sigma(x_\rho)$, hence, can take different forms, depending on the characteristics of the considered slice, but in general it is non-decreasing and such that $f_\sigma(x_\rho) = 1 \forall x_\rho \geq 1$. Therefore, the performance is maximized anytime the allotted resource equals or exceeds the flow request.

Although our framework can accommodate virtually any type of utility function, in this study we considered two functions that we deem suitable to represent eMBB and URLLC classes. For what concerns the eMBB slice, we assume

$$F_e(\mathbf{r}, \hat{\mathbf{r}}) = \alpha_\delta f_e\left(\frac{\delta}{\hat{\delta}}\right) + \sum_{\rho \in \{\eta, c, m\}} \alpha_\rho f_e\left(\frac{\hat{\rho}}{\rho}\right) \quad (1)$$

where $\alpha_\eta, \alpha_c, \alpha_m$, and α_δ are non negative and add up to 1, while $f_e(x_\rho)$ is defined as

$$f_e(x) = \begin{cases} \beta_1 x + \beta_2 x^2 + \beta_3 x^3, & x \in [0, 1]; \\ 1, & x \geq 1. \end{cases} \quad (2)$$

Particularly, β_1, β_2 and β_3 are scalar parameters ensuring that $f_e(\cdot)$ is concave and monotonic increasing for $x \in [0, 1]$. The smooth and concave shape of $f_e(\cdot)$ shown in Fig. 1 reflects the flexibility of the eMBB services with respect to the SLA requirements. Hence, we assume that the quality of experience of the eMBB users degrades rather graciously when the SLA is violated, as in the case of video-streaming applications [29].

Conversely, URLLC flows have very strict requirements that, if infringed, cause the sudden degradation of the related

services. This is the case of applications like, for example, tele-guided robotic surgery, which do not tolerate any increase in the communication delay. For this reason, the performance function for this class of services is defined as the product of step functions $f_u(\cdot)$ (shown in Fig. 1):

$$F_u(\mathbf{r}, \hat{\mathbf{r}}) = f_u\left(\frac{\delta}{\hat{\delta}}\right) \times \prod_{\rho \in \{\eta, c, m\}} f_u\left(\frac{\hat{\rho}}{\rho}\right), \quad (3)$$

where

$$f_u(x) = \begin{cases} 0, & x \in [0, 1); \\ 1, & x \geq 1. \end{cases} \quad (4)$$

In this case, $F_u(\cdot)$ is either maximum, when all the SLA requests are satisfied, or zero when even one single resource requirement is not met.

We remark that our system can be easily extended by defining other slices with different performance functions. Hence, the slice broker can change the composition of the slice set, in order to address the requirements of new tenants in different scenarios.

Given the function $F_\sigma(\cdot)$ for each slice $\sigma \in \Sigma$, the system utility is obtained a weighted sum of the slices performance:

$$\Omega = \frac{|\Phi_e|}{|\Phi|} \Omega_e + \frac{|\Phi_u|}{|\Phi|} \Omega_u, \quad (5)$$

where

$$\Omega_\sigma = \frac{1}{|\Phi_\sigma|} \sum_{\phi \in \Phi_\sigma} F_\sigma(\mathbf{r}_\phi, \hat{\mathbf{r}}_\phi), \quad \sigma \in \Sigma, \quad (6)$$

and $|\mathcal{X}|$ represents the cardinality of $\mathcal{X}$. We observe that $F_\sigma(\cdot)$ always takes values in $[0, 1]$, so that we also have $\Omega \in [0, 1]$.

For completeness, we also define the system utility for a specific type of resource ρ as

$$\Omega^\rho = \frac{|\Phi_e|}{|\Phi|} \Omega_e^\rho + \frac{|\Phi_u|}{|\Phi|} \Omega_u^\rho, \quad (7)$$

where

$$\Omega_\sigma^\rho = \begin{cases} \frac{1}{|\Phi_\sigma|} \sum_{\phi \in \Phi_\sigma} f_\sigma(\rho/\hat{\rho}), & \text{if } \rho = \delta; \\ \frac{1}{|\Phi_\sigma|} \sum_{\phi \in \Phi_\sigma} f_\sigma(\hat{\rho}/\rho), & \text{otherwise;} \end{cases} \quad (8)$$

and $\sigma \in \Sigma$.

B. Network Model

Our model is based on two different network elements, namely *node* and *link*, as detailed below.

- We distinguish two types of nodes: *access nodes* are located at the network edge and connect the users with the rest of the network; *core nodes* are located in the core of the network and forward the aggregate data flows exchanged by the access nodes. We assume that each node n is equipped with a certain amount of computational C_n^c and memory C_n^m resources.
- We call *link* any connection l between two different nodes of the network (fronthaul or backhaul). This element

is provided with a **certain bit rate** B_l to support the communications between the connected nodes.

From now on, we denote by $\mathcal{N}$ and $\mathcal{L}$ the set of network nodes and links, respectively.

In our model, each information flow $\phi \in \Phi$ is initialized between two access nodes (ϵ_ϕ^i and ϵ_ϕ^e) and passes through a certain number of core nodes and links. We observe that, in general, slices can be activated and deactivated on-demand, thus varying the number of flows in the network. Nonetheless, once established, the path of a flow ϕ is maintained for the whole duration of the connection, unless some links become unavailable or the communication end points change. In this case, our model will react as if the flow was interrupted and a new one was started along the new path. We assume that such events are rare and do not impact significantly on the dynamics of the proposed scheme.

We call Φ_l and Φ_n the set of information flows that cross link $l \in \mathcal{L}$ and node $n \in \mathcal{N}$, respectively. Given a link $l \in \mathcal{L}$, each flow $\phi \in \Phi_l$ gets assigned a portion $b_{l,\phi}$ of the link bit rate B_l . Similarly, each node $n \in \mathcal{N}$ assigns to ϕ an amount $c_{n,\phi}$ and $m_{n,\phi}$ of its computational and storage resources. Consequently, any resource allocation pattern must comply with the **following feasibility conditions**:

$$\sum_{\phi \in \Phi_l} b_{l,\phi} \leq B_l, \quad \forall l \in \mathcal{L}; \quad (9)$$

$$\sum_{\phi \in \Phi_n} \rho_{n,\phi} \leq C_n^\rho, \quad \forall n \in \mathcal{N}, \quad \rho \in \{c, m\}. \quad (10)$$

Given a certain allocation of network resources, we want to compute $\hat{r}_\phi$, $\forall \phi \in \Phi$. We denote by $\mathcal{N}_\phi$ and $\mathcal{L}_\phi$ the ordered set of network nodes and links **crossed by** ϕ . In particular, the first and the last element of $\mathcal{N}_\phi$ constitute the flow endpoints. We assume that **the computational and memory requests of a flow $\phi \in \Phi$ can be addressed by aggregating the resources assigned to ϕ by each node along its path**:

$$\hat{\rho}_\phi = \sum_{n \in \mathcal{N}_\phi} \rho_{n,\phi}, \quad \forall \rho \in \{c, m\}. \quad (11)$$

To determine the throughput $\hat{\eta}_\phi$, instead, we need to consider the output flow rate $b_{l,\phi}^o(t)$ at time t from each link $l \in \mathcal{L}_\phi$. Indeed, the throughput corresponds to the output rate from the last link λ along the path:

$$\hat{\eta}_\phi(t) = b_{\lambda,\phi}^o(t). \quad (12)$$

In turn, $b_{l,\phi}^o(t)$ depends on both the bit rate $b_{l,\phi}(t)$ assigned to ϕ by l , the input flow rate $b_{l,\phi}^i(t)$ from the upstream link, and the amount of data of ϕ queued at node n at the end of the previous slot, which is denoted by $D_{l,\phi}(t-1)$. The **input flow rate** $b_{l,\phi}^i(t)$ is given by

$$b_{l,\phi}^i(t) = \begin{cases} b_{\ell,\phi}^o(t), & \text{if } \ell \text{ is the upstream link of } l \text{ in } \mathcal{L}_\phi; \\ \eta_\phi(t), & \text{if } l \text{ is the first link in } \mathcal{L}_\phi. \end{cases} \quad (13)$$

The **output flow** rate $b_{l,\phi}^o(t)$ is then given by the minimum between the allocated rate $b_{l,\phi}(t)$ and the sum of the incoming

and queued traffic, i.e.,

$$b_{l,\phi}^o(t) = \min \left\{ b_{l,\phi}(t), \frac{D_{l,\phi}(t-1)}{T} + b_{l,\phi}^i(t) \right\}. \quad (14)$$

The variable $D_{l,\phi}(t)$ is set to 0 for any time t before the initialization of the flow, and then it is updated as

$$D_{l,\phi}(t) = \max \{0, D_{l,\phi}(t-1) + T(b_{l,\phi}^i(t) - b_{l,\phi}(t))\}. \quad (15)$$

The above equation reflects the fact that the queue length varies in time proportionally to the difference between incoming flow rate $b_{l,\phi}^i$ and assigned link rate $b_{l,\phi}$, growing when this difference is positive and decreasing when it is negative (until the queue is empty).

For what concerns the **delay** experienced by ϕ , we have

$$\hat{\delta}_\phi(t) = \sum_{n \in \mathcal{N}_\phi} \tau_n + \sum_{l \in \mathcal{L}_\phi} \tau_{l,\phi}(t), \quad (16)$$

where τ_n is a positive value representing the delay due to **routing operations at node n** , and it is assumed constant over time. Instead, $\tau_{l,\phi}(t)$ is computed as

$$\tau_{l,\phi}(t) = \tau_{l,\phi}^q(t) + \tau_{l,\phi}^\tau(t) + \tau_l^p, \quad (17)$$

where $\tau_{l,\phi}^q$, $\tau_{l,\phi}^\tau$ and τ_l^p represent the **queuing, transmission, and propagation delays** of ϕ through link l , respectively. In particular, $\tau_{l,\phi}^q(t)$ is given by (see Appendix)

$$\tau_{l,\phi}^q(t) = \frac{2D_{l,\phi}(t-1) - T(b_{l,\phi}^i(t) - b_{l,\phi}^o(t))}{2b_{l,\phi}(t)}, \quad (18)$$

while, $\tau_{l,\phi}^\tau(t)$ is

$$\tau_{l,\phi}^\tau(t) = \frac{1}{b_{l,\phi}^o(t)}. \quad (19)$$

Finally, τ_l^p is a positive and constant value that depends on the physical characteristic of the communication link. We highlight that, despite we consider a discrete time-frame, $\hat{\delta}_\phi$ can take any real (non negative) value.

Our aim is to determine the resource allocation that maximizes the system utility given in (5). Mathematically, we want to determine $\hat{r}_\phi$, $b_{l,\phi}$, $c_{n,\phi}$, $m_{n,\phi}$, $b_{l,\phi}^i$, $b_{l,\phi}^o$, $\tau_{l,\phi}^q$, and $\tau_{l,\phi}^\tau$, $\forall \phi \in \Phi$, $n \in \mathcal{N}$, $l \in \mathcal{L}$ that **maximize Ω** , under the constraints given in (9)-(19).

The many constraints and the non-convexity of Ω make the problem very complex to solve. To determine the optimal solution, a centralized controller would need to have a complete and up-to-date view of all system variables at each timeslot, which is itself challenging. Assuming such complete information is available, conventional optimization tools or meta heuristic techniques could be used to identify the best resource allocations scheme. However, the first may converge on local maxima, while the latter can be unable to find a solution if the resource demands of the information flows evolve quickly in time. These issues can be alleviated by distributed control algorithms that can promptly take new actions, albeit with a partial view on the overall network. The DRL paradigm is suitable for this problem since it can **provide high-performance solutions to carry our complex control tasks also when the environment is partially observable.**

IV. LEARNING STRATEGY

In order to efficiently orchestrate communication resources in a NS scenario, we develop a **distributed architecture** based on **multiple learning units**, named **local controllers**, that **collaborate to maximize the overall system utility** given by (5). In the rest of the section, we will recall the main principles of DRL, and present the framework used to train the learning agents of our architecture.

A. Deep Reinforcement Learning

The RL paradigm is one of the main branches of ML. Particularly, RL does not solely aim at solving classification or regression problems, but it enables the development of complex strategies to maximize the long-term performance of a target system [30]. Moreover, RL algorithms do not need to have labeled data to carry out the training phase, but they interact with an environment where agent actions are associated to specific rewards.

In a RL scenario, the target system is modeled as a Markov Decision Process (MDP), which is a powerful mathematical tool used to represent decision-making problems [31]. This framework requires to define a state space $\mathcal{S}$ of the environment, an action space $\mathcal{A}$ of the learning agent, and a reward function $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. During any timeslot t , the learning agent observes the system state $s_t \in \mathcal{S}$, performs an action $a_t \in \mathcal{A}$ and receives a reward $r_t \in \mathbb{R}$. Hence, the future state s_{t+1} depends uniquely on the previous state s_t and the agent decision a_t .

The agent chooses new action according to a policy $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$, where $\pi(s, a)$ is the probability to take the action a from state s . The aim of any RL algorithm is to determine a policy that maximizes the *discounted return*, defined as

$$G_t = \sum_{k=0}^{\infty} \lambda^k r_{t+k}, \quad (20)$$

where $\lambda \in (0, 1)$ is the so-called *discount factor*. Particularly, if $\lambda \rightarrow 0$, the algorithm will behave in a myopic manner, taking actions that can acquire high reward in the short term; instead, if $\lambda \rightarrow 1$, the algorithm will determine the actions that bring more benefit in the long term.

Given a policy π , an RL algorithm associates each possible state s with a value $V_\pi(s)$. The function $V_\pi(\cdot)$ is called *state value function* and represents the expected discounted return that is achieved following the strategy π from state s :

$$V_\pi(s) = E[G_t | s, \pi]. \quad (21)$$

As the agent explores the learning environment, the values of $V_\pi(s)$ and the policy itself are updated. In particular, the *Bellman optimality equation* ensures that the optimal policy π^* provides the maximum value of $V_\pi(s) \forall s \in \mathcal{S}$, i.e.,

$$V_{\pi^*}(s) = \max_{\pi} V_\pi(s), \quad \forall s \in \mathcal{S}. \quad (22)$$

If the state and action spaces get too complex, conventional RL approaches may fail to determine the optimal policy because of the *curse of dimensionality* [32]. To address such a problem, the DRL paradigm makes use of deep NNs to

estimate the state value function and the optimal policy. In particular, DRL algorithms are capable of handling continuous state and action spaces, which means that $|\mathcal{S} \times \mathcal{A}| \rightarrow \infty$.

B. Learning Architecture

In this work, we adopt an Actor Critic (AC) approach [33], which involves the learning of the optimal policy by two different units. The first is named *actor* and approximates the optimal policy π_θ , parameterized by θ ; the latter is named *critic* and approximates the value function V_γ , parameterized by γ . Hence, during any timeslot t , the actor determines the action a_t that the policy π_θ takes in the current state s_t . Instead, the critic computes the expected discounted reward that is obtained following the policy π_θ from s_t .

To carry out the system training, we exploit the Advantage Actor Critic (A2C) algorithm, which has shown to provide stable DRL solutions in very complex scenarios [34]. The critic is trained to minimize the function $L_c(s_t, a_t) = A(s_t, a_t)^2$, where

$$A(s_t, a_t) = r_t + \lambda V_\gamma(s_{t+1}) - V_\gamma(s_t). \quad (23)$$

Particularly, the function $A : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is called *advantage* and represents the benefit obtained by choosing action a_t rather than the one recommended by the current policy π . Conversely, the actor is trained to minimize the function

$$L_a(s_t, a_t) = -\nabla_\theta \log \pi_\theta(s_t, a_t) A(s_t, a_t)^2 - \kappa H(\pi_\theta), \quad (24)$$

where ∇_θ is the gradient with respect to θ , $H(\pi_\theta)$ is the entropy of π_θ , and κ is a scalar value. As suggested in [34], the actor loss function depends linearly on the policy entropy $H(\pi_\theta)$. Particularly, we can promote the exploration of the action space by increasing κ since, in such a case, the actor gain benefits to take random actions. Instead, as $\kappa \rightarrow 0$, the actor will choose actions that are expected to bring the highest reward according to the current experience.

We highlight that A2C enables to consider continuous action spaces: this is not possible with traditional RL algorithms (e.g., Q-Learning) that, instead, can only take actions from discrete sets. Besides, the A2C algorithm supports an on-policy training and, consequently, allows the agents to continuously refine the target policy while this is used in the real system. Hence, the slice broker can progressively and dynamically adapt the policy to new scenarios, without needing to train a new system from scratch every time the working conditions (e.g., target network topology or slice requirements) change.

Our learning architecture (shown in Fig. 2) provides a different controller for each information flow and network element. From an operational point of view, the total number of local controllers depends on the network topology and the cardinality of Φ . Practically, the local controllers are all replicas of $3 \times |\Sigma|$ learning agents. During the training phase, we design a tuples of agents $(\Gamma_\sigma^b, \Gamma_\sigma^c, \Gamma_\sigma^m)$ for each different slice $\sigma \in \Sigma$: the agent Γ_σ^b is trained to orchestrate the bit rate of each information flow $\phi \in \Phi_\sigma$ in each link $l \in \mathcal{L}$; instead, Γ_σ^c and Γ_σ^m are trained to orchestrate the computation and memory resources of each information flow $\phi \in \Phi_\sigma$ in each node $n \in \mathcal{N}$.

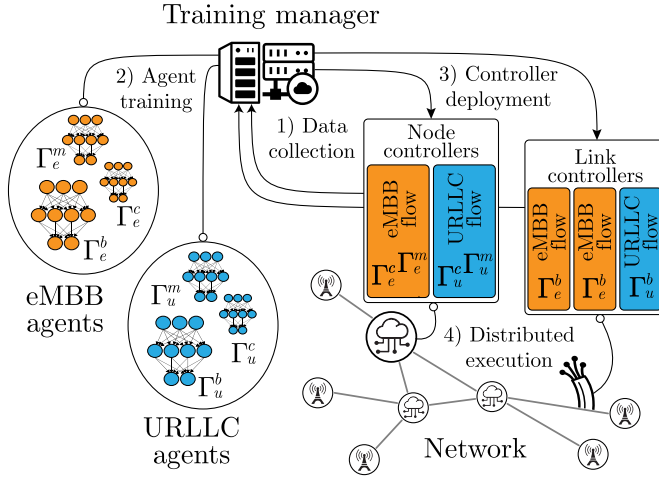


Fig. 2. Learning architecture.

As shown in Fig. 2, the training phase is performed by a central entity, named *training manager*, which collects the system information and updates the learning agents accordingly. Then, copies of Γ_e^b , Γ_e^c , and Γ_e^m will be deployed in each network element crossed by any flow $\phi \in \Phi_\sigma$. Alternatively, each network element may develop its own version of the learning agents, using only local information and avoiding interactions with the central manager; the drawback is that the training phase will lack generality. In Sec. VI-C, we will show how the transfer learning paradigm can be used to combine the two approaches.

According to the A2C algorithm, each agent is composed by two units, i.e., the actor and the critic, which are implemented by means of NNs. Particularly, we consider an architecture with two hidden layers and the Rectifier Linear Unit (ReLU) as activation function [35]. The output of the actor is the amount ρ^* of resources that the local controller demands, while that of the critic is the expected future reward. The size of the NN input varies according to the resources that have to be managed, as explained in the next subsection.

C. Observations and Actions

In our system, each local controller has full knowledge of the element where it is installed, while it has a limited view on the network, which implies that the system state is only partially observable [36]. Let us consider a local controller managing the rate resources of a flow ϕ in a link l . At the beginning of each timeslot, such a controller is provided with two vectors representing the status of the information flow and the network element it is associated with.

The first vector gives the state of ϕ at the beginning of timeslot t :

$$\mathbf{s}_\phi(t) = [\mathbf{r}_\phi(t), \hat{\mathbf{r}}_\phi(t-1)], \quad (25)$$

where $\mathbf{r}_\phi(t)$ and $\hat{\mathbf{r}}_\phi(t-1)$, defined in Sec. III, are the resources requested by and granted to ϕ during timeslot t and $t-1$, respectively. We observe that $\mathbf{r}_\phi(t)$ and $\hat{\mathbf{r}}_\phi(t-1)$ can be computed only knowing the aggregate amount of network resources assigned to ϕ by the network elements across its routing path. Therefore, $\mathbf{s}_\phi(t)$ needs to be shared among all

the controllers assigned to ϕ at the beginning of each timeslot t . However, the size of $\mathbf{s}_\phi(t)$ is negligible with respect to the rate requirements of the slices, and, therefore, can be transmitted within the user data plane of ϕ , without degrading the performance of our system.

The second vector provides the state of the rate resources of ϕ in l at the beginning of timeslot t :

$$\mathbf{s}_{l,\phi}(t) = [B_l, \tau_{l,\phi}(t-1), D_{l,\phi}(t-1), b_{l,\phi}^*(t-1), \\ \times b_{l,\phi}(t-1), b_l^e(t-1), b_l^u(t-1)], \quad (26)$$

where $b_l^\sigma(t-1)$ is the aggregate rate demanded at the link l by all flows of class σ during timeslot $t-1$, while the other parameters were defined in Sec. III. We highlight that this information is locally available to the link controller and, consequently, $\mathbf{s}_{l,\phi}(t)$ does not need to be transmitted across the network. Hence, at the beginning of timeslot t , the controller takes $\mathbf{s}_\phi(t)$ and $\mathbf{s}_{l,\phi}(t)$ as input and returns $b_{l,\phi}^*(t)$, which is the bitrate requested by ϕ at link l during timeslot t .

When considering a controller associated to a node n and a flow ϕ we use the same approach and, depending on the resource $\rho \in \{c, m\}$ we want to allocate, we substitute (26) with $s_{n,\phi}^c(t)$ or $s_{n,\phi}^m(t)$, which are the states of the computation and memory resources assigned to ϕ by node n . In particular, $s_{n,\phi}^c(t)$ or $s_{n,\phi}^m(t)$ are defined as

$$\mathbf{s}_{n,\phi}^\rho(t) = [C_n^\rho, \rho_{n,\phi}^*(t-1), \rho_{n,\phi}(t-1), \\ \times \rho_n^e(t-1), \rho_n^u(t-1)], \quad (27)$$

where $\rho_n^\sigma(t-1)$ is the aggregate amount of resource ρ demanded to node n by all the flows of class σ during timeslot $t-1$. As before, the node controller takes $\mathbf{s}_\phi(t)$ and $\mathbf{s}_{n,\phi}^\rho(t)$ as input and returns $\rho_{n,\phi}^*(t)$, with $\rho \in \{c, m\}$, which is the amount of computational (or memory) capacity demanded by ϕ to node n during timeslot t .

D. Reward Function

In accordance to the RL paradigm, we need to define a reward function $r(\cdot)$ that represents the benefit generated by each possible state-action pair of the policy. In particular, to maximize the overall utility, each local controller should demand enough resources to maximize the performance of the flow ϕ it is in charge of, without subtracting too many resources to the other flows. In our system, given a controller associated to a link $l \in \mathcal{L}$ and a flow $\phi \in \Phi_l$, the reward at time t is given by

$$r_{l,\phi}(t) = \gamma_0 \left(f_{\sigma_\phi} \left(\frac{\eta_\phi(t)}{\hat{\eta}_\phi(t)} \right) + f_{\sigma_\phi} \left(\frac{\hat{\delta}_\phi(t)}{\delta_\phi(t)} \right) \right) \\ + \frac{\gamma_1}{|\Phi_l|} \sum_{\psi \in \Phi_l} \left(f_{\sigma_\psi} \left(\frac{\eta_\psi(t)}{\hat{\eta}_\psi(t)} \right) + f_{\sigma_\psi} \left(\frac{\hat{\delta}_\psi(t)}{\delta_\psi(t)} \right) \right), \quad (28)$$

where σ_ϕ is the slice class that ϕ belongs to, while γ_0 and γ_1 are positive scalar values. In particular, γ_0 weights the throughput and delay performance of flow ϕ , while γ_1 weights the average throughput and delay performance of all the other flows crossing link l .

Similarly, a controller assigned to a node n and a flow ϕ is rewarded according to

$$r_{n,\phi}^\rho(t) = \gamma_0 f_{\sigma_\phi} \left(\frac{\rho_\phi(t)}{\hat{\rho}_\phi(t)} \right) + \frac{\gamma_1}{|\Phi_n|} \sum_{\psi \in \Phi_n} f_{\sigma_\psi} \left(\frac{\rho_\psi(t)}{\hat{\rho}_\psi(t)} \right), \quad (29)$$

where $\rho \in \{c, m\}$, while the scalar values γ_0 and γ_1 have the same role as before.

Therefore, the reward function consists of the weighted sum of two terms: the first reflects the performance of the flow targeted by the controller, while the second represents the aggregate performance of all the flows crossing the considered network element. With such a mixed reward function, the controller will hence attempt to improve the quality of the targeted flow, but without unduly penalizing other flows.

V. BENCHMARK STRATEGIES

In this section, we discuss some DRL approaches that can be considered to benchmark the proposed solution, starting from some of the literature works described in Sec. II. Then, we define two baseline strategies: the first is an **empirical algorithm**, which tries to fairly allocate communication and processing resources in each network allocation, while the latter is based on **meta-heuristic optimization** and performs a static allocation of network resources.

We highlight that the chosen benchmarks make use of information that is not supposed to be available to the DRL approach, thus gaining a competitive advantage over it. This choice permits a simple interpretation of the results, highlighting the pros and cons of our DRL system without being trivial.

A. Other Learning-Based Approaches

Out of the several ML algorithms described in Sec. II, the approaches proposed in [23]–[27] are those more aligned with the problem addressed in this paper. Although fundamental differences in the system model prevent a direct comparison between such approaches and our solution, some high level considerations are still possible.

We first observe that the authors of [23]–[25] focus on the basic scenario of coordinating the users associated with a unique cell, focusing on one specific type of resource (namely, channel bandwidth). These solutions are not well suited to handle the allocation of multiple resources, which affect different performance indices in an intertwined manner. In our system, such methods will fail, not being designed to deal with the reward variability determined by the interaction among multiple agents deployed in different network locations.

The authors of [26] design a multi-node topology but assume that a single centralized agent can operate in the network, considering one node at a time. This approach does not suit the high dynamism of our scenario, which requires reallocating bandwidth and computational power in multiple network nodes and links simultaneously. In particular, if the demands of network resources change too fast in time, the resources assigned to the different slices may be insufficient or unnecessary. On the other hand, our system can well suit the scenario presented in [26], allowing a faster allocation at

the cost of a sub-optimal performance since it is not possible for the learning agents to observe the entire network status.

The framework proposed in [27] is the only one involving a multi-agent system to orchestrate the slice resources, in line with what is proposed in our manuscript. However, in [27], the different agents can share their views on the network status in real-time, which violates one of the fundamental hypotheses of our model. To make such a strategy consistent with our system model, we should consider the delay in the computation of the embedding vectors, which makes it impossible for the agents to estimate the network status in real-time. At the same time, implementing our approach in the model of [27], we expect to obtain a degraded performance since each agent or our system exploits only one part of the network information available.

We finally highlight that none of the above algorithms can adapt to dynamic network topologies, which is one of the most important aspects of our multi-agent framework. Implementing any of these approaches in our system model requires the complete redesign of the solutions, making it impossible to carry out a fair analysis. For this reason, in the rest of the paper, we will compare our DRL architecture with two simple algorithms, based on easy to interpret criteria, that are described in what follows.

B. Empirical Strategy

Similarly to our approach, the **empirical strategy implements a distributed resource allocation scheme**. At the beginning of timeslot t , each flow ϕ crossing link l demands a bit rate sufficient to both satisfy the current throughput requirement and transmit any buffered data, i.e.,

$$b_{l,\phi}^*(t) = \eta_\phi(t) + \frac{D_{l,\phi}(t-1)}{T}. \quad (30)$$

For what concerns the computation and memory allocation, each flow ϕ **distributes its requests among the nodes along its path, proportionally to their capacity**. More specifically, the amount of resources of type $\rho \in \{c, m\}$ required to node n is equal to

$$\rho_{n,\phi}^*(t) = \chi_{n,\phi} \rho_\phi(t), \quad (31)$$

with

$$\chi_{n,\phi} = \frac{C_n^\rho}{\sum_{k \in \mathcal{N}_\phi} C_k^\rho}. \quad (32)$$

We highlight that, to compute $\chi_{n,\phi}$, it is necessary to know the computation and storage capacities of each node $n \in \mathcal{N}_\phi$, an information that is not assumed to be available to the controllers in our DRL strategy. Hence, the empirical strategy has an advantage with respect to our learning framework since it uses information that is generally not available in a fully distributed approach.

We observe that, either using the empirical or the DRL strategy, network elements may not be able to satisfy all the requests they receive. Particularly, the total aggregate amount of resources that is demanded to a link l (or a node n) **may exceed its overall capacity**. Hence, we need to map the demanded resources $b_{l,\phi}^*, c_{n,\phi}^*, m_{n,\phi}^*$ to the assigned resources $b_{l,\phi}, c_{n,\phi}, m_{n,\phi}, \forall l \in \mathcal{L}, n \in \mathcal{N}$, **ensuring that the feasibility constraints (9) and (10) are always satisfied**.

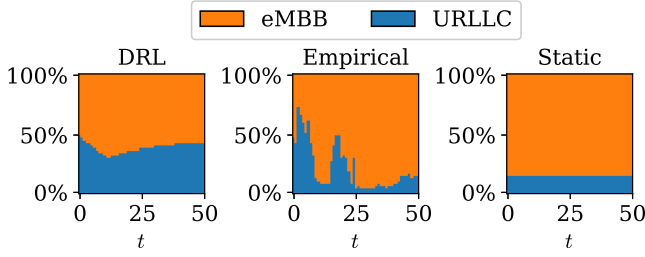


Fig. 3. Link resource allocation.

Let us consider a link $l \in \mathcal{L}$ during a timeslot t . If the total demand is lower than B_l , the feasibility constraints are already met: consequently, we can set $b_{l,\phi}(t) = b_{l,\phi}^*(t)$. Conversely, if the aggregate request exceeds the link capacity B_l , then each flow gets at most a fraction of the link rate proportional to its request:

$$b_{l,\phi}(t) = b_{l,\phi}^*(t) \min \left\{ 1, \frac{B_l}{\sum_{\psi \in \Phi_l} b_{l,\psi}^*(t)} \right\}. \quad (33)$$

Using the same principle for the computational and memory resources, we can write

$$c_{n,\phi}(t) = c_{n,\phi}^*(t) \min \left\{ 1, \frac{C_n^c}{\sum_{\psi \in \Phi_n} c_{n,\psi}^*(t)} \right\}, \quad (34)$$

and

$$m_{n,\phi}(t) = m_{n,\phi}^*(t) \min \left\{ 1, \frac{C_n^m}{\sum_{\psi \in \Phi_n} m_{n,\psi}^*(t)} \right\}. \quad (35)$$

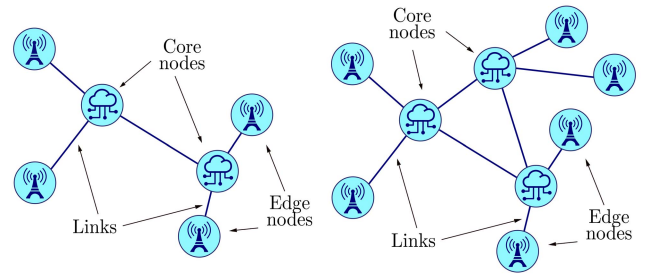
C. Static Strategy

Meta-heuristic techniques have been shown to determine the optimal solution of highly complex optimization problems with non-convex constraints [37]. In our system, a Genetic Algorithm (GA) may be used to approximate the best resource allocation pattern, thus outperforming both the DRL and empirical strategies. However, meta-heuristic algorithms are based on a randomized search of the target solution and require an extremely long calculation time, which makes it unfeasible to execute them at each timeslot. At the same time, it is reasonable to exploit meta-heuristic techniques if the resource requirements do not vary in time.

In the static strategy we consider in this work, the network resources are statically divided among the different slices. Practically, each traffic flow ϕ is assumed to have fixed requirements, corresponding to the average amount of resources it demands, i.e., $\mathbf{r}_\phi^a$. Hence, a GA is used to determine the optimal resource allocation pattern under such conditions [38].

We observe that, using the static strategy, the values of $b_{l,\phi}$, $c_{n,\phi}$, $m_{n,\phi} \forall \phi \in \Phi$, $l \in \mathcal{L}$, $n \in \mathcal{N}$ are maintained fixed. In other words, the variability of information flows is not taken into account, and the performance of each flow is expected to deteriorate as soon as its requirements exceed the average values. This is a big issue, especially for the URLLC slice, whose performance function suddenly drops if any resource requirements is not satisfied.

To better highlight the characteristics of the different strategies, in Fig. 3, we represent the share of resources assigned



(a) Dumbbell Network (D).

(b) Triangle Network (T).

(c) Pyramid Network (P).

Fig. 4. Network topologies.

to the eMBB and URLLC services in a network link during a period of 50 timeslots. In particular, we consider a scenario where the link capacity is lower than the aggregate resource demand, which means that all the strategies fully use the available bitrate. As expected, GA distributes network resources in a static fashion: the bitrate assigned to the different slices does not vary in time. Instead, the empirical algorithm dynamically adapts to the slice requirements and, hence, the bitrate distribution changes at each timeslot. Also the DRL strategy follows a dynamic trend but it assigns a larger amount of resources to the URLLC services than the benchmarks.

VI. SIMULATION SETTING AND RESULTS

In this section, we first describe the scenarios where our algorithms are tested as well as the setting of our simulations, whose parameters are summarized in Tab. II. Then, we investigate the performance of our DRL architecture against the benchmark strategies and under different conditions. Finally, we show how the transfer learning paradigm can be used to further improve the system performance.

A. Setting

We consider three different network scenarios, named *Dumbbell* (D), *Triangle* (T), and *Pyramid Network* (P), whose topologies are reported in Fig. 4. In all the cases, the number of information flows in the network is $N_\Phi \in \{2, \dots, 6\}$. The capacities of each network element are fixed; specifically, we set $B_l = 50$ Gbps, $C_n^c = 60$ Gbps and $C_n^m = 60$ Gb for the core nodes, $C_n^c = 20$ Gbps and $C_n^m = 20$ Gb for the access nodes. Hence, we consider that most computing and storage resources are concentrated in the core network. Concerning the delay, we assume that $\tau_l^p = 0.1$ ms, $\forall l \in \mathcal{L}$, and $\tau_n = 0.001$ ms, $\forall n \in \mathcal{N}$. Although arbitrary, these values are well-aligned with the features of modern network elements.

TABLE II
SIMULATION SETTINGS

Parameter	Value	Description	Parameter	Value	Description
B_l	50 Gbps	Link rate capacity	C_n^c	{10, 20, 30, 60} Gbps	Node computational capacity
C_n^m	{10, 20, 30, 60} Gb	Node memory capacity	N_{state}	10	State number
τ_n	0.001 ms	Node routing delay	τ_l^p	0.1 ms	Link propagation delay
N_{train}	{3, 5} · 10 ⁴	Training episodes	N_{test}	500	Testing episodes
$N_{transfer}$	2 · 10 ⁴	Transfer learning episodes	N_Φ	{2, ..., 6}	Number of flows
ζ_a	10 ⁻⁵	Actor learning rate	ζ_c	10 ⁻⁵	Critic learning rate
λ	0.9	Discount factor	κ	10 ⁻⁴	Entropy weight
T	0.1 s	Timeslot duration	N_{slot}	50	Timeslots per episode
$\alpha_\eta, \alpha_c, \alpha_m, \alpha_\delta$	0.25, 0.25, 0.25, 0.25	eMBB performance weights	γ_0, γ_1	0.1, 1	Reward weights

TABLE III
TRAFFIC FLOW REQUIREMENTS

Service class	Parameter	Range of values	Unit	Sources
eMBB	η	0.30 ÷ 42.5	Gbps	[39], [40]
	c	50 ÷ 100	Gbps	[41], [42]
	m	50 ÷ 100	Gb	[41], [42]
	δ	20	ms	[43]–[46]
URLLC	η	2.08 ÷ 10	Gbps	[39], [40]
	c	50 ÷ 100	Gbps	[41], [42]
	m	50 ÷ 100	Gb	[41], [42]
	δ	1	ms	[43]–[46]

To model the information flow requirements, we consider a Markov Model (MM) [47] M_σ with transition probability matrix $\mathbf{P}_\sigma$ for each slice $\sigma \in \Sigma$. The model include $N_{state} = 10$ states and is designed in such a way that transitions can only occur between adjacent states; mathematically, $\mathbf{P}_{i,j} = 0 \forall (i, j) : |i-j| > 1$. Each state of M_σ represents a combination of resources requirements, i.e., a different realization of the vector $\mathbf{r}$. Hence, each information flow $\phi \in \Phi_\sigma$ is associated to an independent copy of M_σ that changes state at each timeslot t , thus altering the vector $\mathbf{r}_\phi$. The minimum and maximum value of the resource requirements are given in Tab. III.

To be noted that, in our simulations, the number of flows and their requirements change randomly, so that the aggregate resource requests can exceed the capacity of the network. In such conditions, some flows will unavoidably experience very low performance and the allocation strategy should decide which flow to penalize, in order to maximize the overall utility. Therefore, our system can be exploited to handle critical scenarios where there is a lack of network resources, or to estimate the reliability of a specific set of network slices. In the future, we will investigate call-admission control strategies to avoid system overloading.

To train the learning agents, we generate $N_{train} = 5 \cdot 10^4$ independent episodes using the same network topology. Each episode lasts $N_{slot} = 50$ timeslots of $T = 0.1$ seconds. At the beginning of each episode, a random number of information flows is generated and each flow is associated with a static route interconnecting its endpoints. Then, the A2C algorithm is used to train the learning agents Γ_σ^b , Γ_σ^c , Γ_σ^m , $\forall \sigma \in \Sigma$, according to the framework describe in Sec. IV. We exploit the Adaptive moment estimation (Adam) algorithm to optimize the NN weights, considering $\zeta_a = 10^{-5}$ and $\zeta_c = 10^{-5}$ as learning rates of the actor and the critic, respectively. The settings of the learning architectures are given in Tab. IV.

TABLE IV
AGENT ARCHITECTURES

Parameter	Γ_σ^b		Γ_σ^c		Γ_σ^m	
	Actor	Critic	Actor	Critic	Actor	Critic
Input size	11	11	7	7	7	7
Activation	ReLU	ReLU	ReLU	ReLU	ReLU	ReLU
Hidden size	12	12	8	8	8	8
Activation	ReLU	ReLU	ReLU	ReLU	ReLU	ReLU
Hidden size	6	6	4	4	4	4
Activation	Linear	Linear	Linear	Linear	Linear	Linear
Output size	1	1	1	1	1	1

To be noted that the A2C algorithm updates the policies applied by the different agents only at the end of each episode (i.e., after a predetermined number of timeslots). In the period between two subsequent episodes, the different network elements forward local observations to the training manager depicted in Fig. 2. Hence, the new versions of the learning agents is shared within the network, and the overall learning architecture is updated consequently. Besides, the exchange of control traffic among network elements is intrinsic to the SDN and NFV paradigms, which are becoming more and more widespread in modern networks. Therefore, the communication within the learning units does not represent a limiting factor for the practical implementation of our approach.

B. Results

We consider two versions of our learning system, one trained in the Dumbbell Network (DRL-D) and the other in the Triangle Network (DRL-T). In Fig. 5 we plot the utility (for the eMBB and URLLC slices) obtained during the training phase in the two scenarios, which makes it possible to assess the convergence time of the proposed framework.

In general, the convergence of a ML algorithm is determined by the slope of the *learning curve*, which is given by the loss function of the trained model. In a DRL scenario, the convergence does not depend solely on the training loss but also on the *Q-values* associated to the different actions. Indeed, if the hierarchy between the Q-values does not change in time, the agent policy remains static, regardless of the learning process. To assess the convergence of a DRL algorithm, it is hence more accurate to verify when the distribution of the agent actions becomes invariant, which implies that also the agent reward and the system utility have been stabilized.

In this work, we assumed convergence was reached when the utility stabilizes, which means that any variation in the

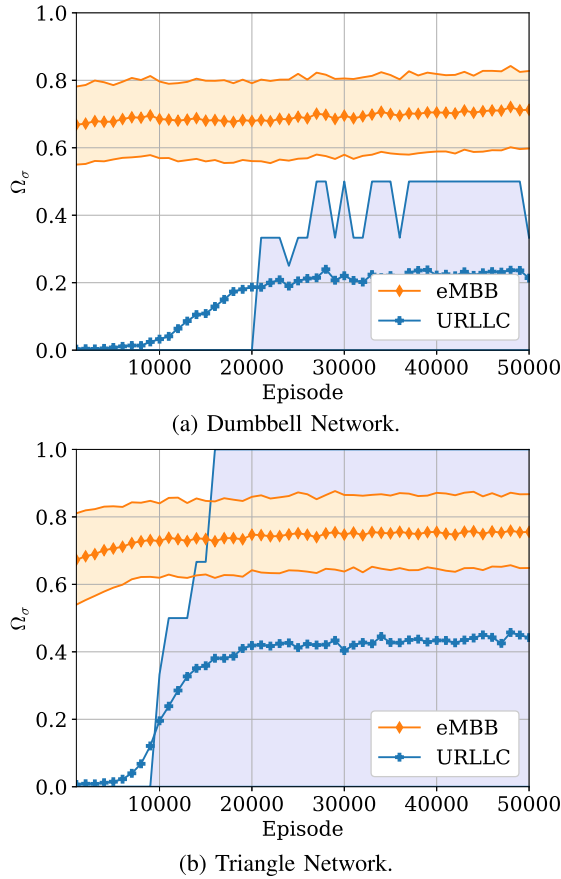


Fig. 5. Training phase.

agents' actions does not lead to a better policy. We can appreciate that the average and the percentiles of the slice performance stop increasing after 30000 training episodes, which indicates that the agents have assumed a stable behavior after that point. This is confirmed also by the trend of the algorithm reward and loss function, whose plots are omitted for space constraints. We remark that the convergence of DRL systems cannot be proved mathematically, unless under specific constraints. In particular, more complex scenarios (e.g., with a larger network topology) may exhibit longer convergence times. Therefore, network designers have to accurately tune the number of training episodes case by case.

Looking at Fig. 5, we can observe that the performance of the eMBB slice (Ω_e) increases slowly but smoothly, ranging from 0.6 to 0.8 in 50% of the cases. Conversely, the performance of the URLLC slice (Ω_u) initially remains very low and suddenly increases after a certain number of episodes. This means that the agents need more time to learn how to address the requirements of the URLLC services. In particular, at the end of the training phase in the Triangle Network, Ω_u spans the full range of possible values, which indicates that the URLLC flows either get maximum or zero reward in accordance with the step-like shape of their performance function. A similar phenomenon occurs in the Dumbbell Network where, however, the URLLC utility is lower due to the fewer available resources.

To test the performance of our strategies, we carry out additional $N_{\text{test}} = 500$ episodes. In Fig. 6, we report the expected

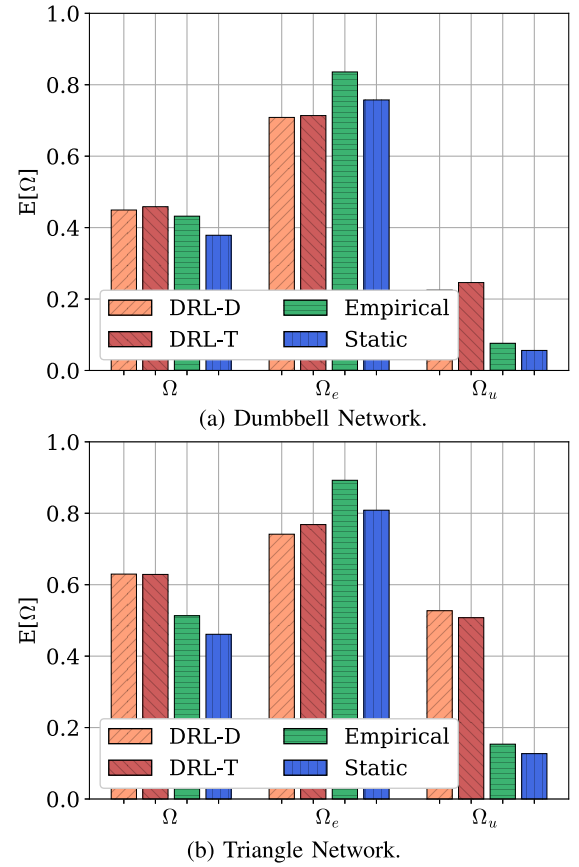


Fig. 6. Expected utility.

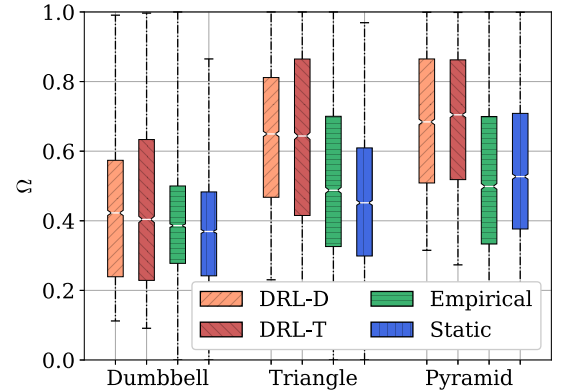


Fig. 7. Utility distribution.

utility achieved by each slice, and by the whole system (see (6) and (5)), in the Dumbbell and Triangle Network for all the considered strategies. We can notice that, in both scenarios, the empirical algorithm tends to favor eMBB slices at the expenses of URLLC flows, whose expected performance is always lower than 0.2. The static strategy behaves similarly but provides lower Ω_e than the empirical algorithm. In contrast, DRL-D and DRL-T slightly reduce the performance of the eMBB services but double the fraction of satisfied URLLC flows, significantly increasing the total utility.

In Fig. 7 we analyze the distribution of the system utility for all the different strategies and network scenarios. We adopt the

boxplot representation, where the white line at the center of the box is the median of the distribution, while the box edges represent the 25th and the 75th percentile, respectively. It is clear that the static strategy always performs the worst because it cannot handle the variability of service demand, which is very critical for URLLC traffic flows, whose performance goes to zero as soon as one requirement is not satisfied. The empirical algorithm works better but is outperformed by the DRL strategies. In the Dumbbell Network, DRL-D and DRL-T ensure $\Omega > 0.4$ in almost 50% of the test episodes, with a 10% gain over the empirical algorithm. In the Triangle and Pyramid scenario, the lack of network resources is less striking and, consequently, the performance of all the strategies increases. In these scenarios, using the empirical algorithm, in 50% of the test episodes we get $\Omega > 0.5$ while, with the DRL strategies, we achieve $\Omega > 0.65$ and $\Omega > 0.7$, respectively.

We observe that the DRL strategies achieve similar results in all the testing scenarios, including the Pyramid Network, which is a different environment from those seen during the learning phase. Therefore, the proposed distributed solution seems to be rather robust to topology changes, being able to yield reasonably good results in different scenarios without the need for additional training. At the same time, we expect that a more specific learning framework can further improve the overall utility. In the rest of the section, we will show how to leverage the transfer learning paradigm to this purpose.

C. Transfer Learning

Transfer learning aims at speeding up the training of a ML algorithm by taking advantage of the structure learned by other algorithms trained in similar scenarios. In what follows, we exploit this technique to adapt our DRL strategy to different network topologies and traffic loads. Hence, we consider two additional learning architectures that we name DRL-DP and DRL-TP. These systems are first trained for $N_{\text{train}} = 3 \times 10^4$ episodes in the Dumbbell and Triangle Network, respectively, according to the framework described in Sec. IV. Then, they undergo an additional training of $N_{\text{transfer}} = 2 \times 10^4$ episodes in the Pyramid Network.

During the additional training, each controller is updated only with experience related to the specific network element it is associated to. For instance, a controller Γ_{σ}^b designed to manage slice σ across a link l is trained using the state-action pairs for link l and information flows $\phi \in \Phi_{\sigma}$. This implies that each network elements can autonomously update its own learning agents, without involving the central manager depicted in Fig. 2. Besides, the training process does not require any communication across the network and can be executed online, i.e., while the local controllers are being used in the real system. From a practical perspective, transfer learning makes local controllers learn how to carry out more precise actions, thus increasing the overall utility. The drawback is that each controller will be able to operate only in a specific location and, therefore, the overall architecture cannot be implemented in different network topologies.

The transfer learning operations are repeated two times, considering different configurations for the URLLC services. First, we implement the same statistics presented in Tab. III:

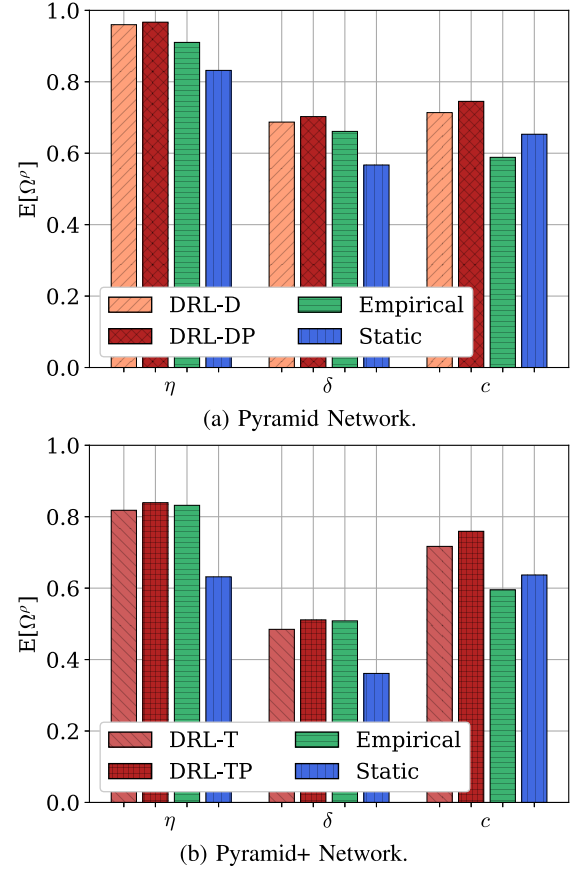


Fig. 8. Expected resource utility.

in this case, the throughput required by each URLLC flow is in $[2.08, 10]$ Gbps. Then, we double the rate requirements of the URLLC slice (whose range of values becomes $[4.16, 20]$ Gbps), with the aim of assessing the ability of our strategy to adapt to new service specifications. In particular, we denote by **Pyramid+** the scenario in which the required throughput of the URLLC flows is increased.

In Fig. 8a, we represent the expected performance obtained in the Pyramid Network by DRL-D and DRL-DP, and the benchmark strategies, while considering specific network resources. Hence, instead of using Ω to assess the system utility, we consider Ω^ρ , with $\rho \in \{b, c, m, \delta\}$, defined in (8). As expected, the algorithm obtained by exploiting transfer learning performs better than that trained in the Dumbbell Network: for instance, using the DRL-DP strategy, the expectation of Ω^c increases by more than 5%. In this case, the performance gain is considerably lower than during the centralized training phase, and the system reaches convergence in about 10000 episodes. The higher speed in the system training is because the agents have already discovered a profitable policy and are just tuning their actions to the new scenario. In particular, the slice performance remains high during the whole transfer learning phase, making it feasible to train the system online without degrading the user experience.

In Fig. 8b, we show the outcomes obtained in the Pyramid+ network scenario, considering DRL-T and DRL-TP. In this scenario, all the strategies yield a lower performance since the new URLLC requirements are more difficult to fulfill.

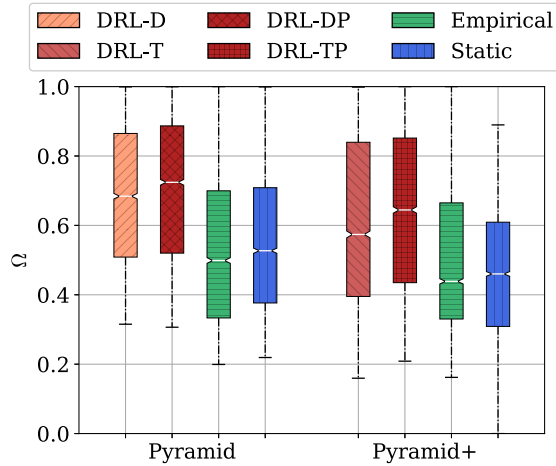


Fig. 9. Utility distribution.

Besides, we observe that the empirical strategy slightly outperforms DRL-T for what concerns the throughput and delay requirements. Nevertheless, DRL-T still provides good results, which means that the DRL approach is resilient to different traffic loads. Also in this case, transfer learning proves to be worthwhile since DRL-TP outperforms the other algorithms in all three key performance indicators.

In Fig. 9, we report the distribution of the system utility in the Pyramid and Pyramid+ scenarios. We can appreciate how the algorithms obtained by transfer learning always ensure higher utility, both considering the median and the percentiles of Ω . In particular, DRL-DP and DRL-TP outperform DRL-D and DRL-T, respectively, despite the total number of training episodes is the same for all the considered strategies.

D. Practical Implementation

To exemplify a possible application of the proposed solution in a real scenario, we consider a section of the high capacity network interconnecting the main Italian universities and research centers owned by the national provider named GARR [48]. As depicted in Fig. 10, the considered network topology includes 19 core nodes, 10 edge nodes and 40 links. We assume that each link is provided with $B_l = 50$ Gbps of bandwidth capacity. Besides, we set the computational and memory capacities of the core nodes to $C_n^c = 30$ Gbps and $C_n^m = 30$ Gb, and those of the edge nodes to $C_n^c = 10$ Gbps and $C_n^m = 10$ Gb.¹ Finally, we relax the delay requirements of the eMBB and URLLC services, which are set to 100 ms and 5 ms, respectively. We make this choice to deal with the longer routing paths, which yield higher end-to-end delays.

To implement our DRL strategy in this scenario, we carry out two consecutive learning stages: first, we train our system for 2×10^4 episodes using a centralized learning approach (as done for the DRL-D and DRL-T strategies). We recall that this phase can be (and shall be) performed offline, using all the experience gathered by all the network elements. Then, we carry out an additional training of 10^4 episodes, defining

¹Note that, currently, the GARR network does not support NFV or SDN functionalities, so that the resource capabilities considered in this study are realistic, but arbitrary.

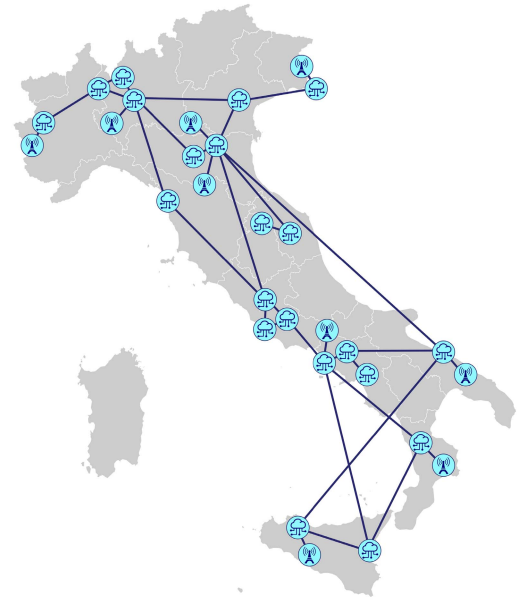


Fig. 10. GARR network (Italy).

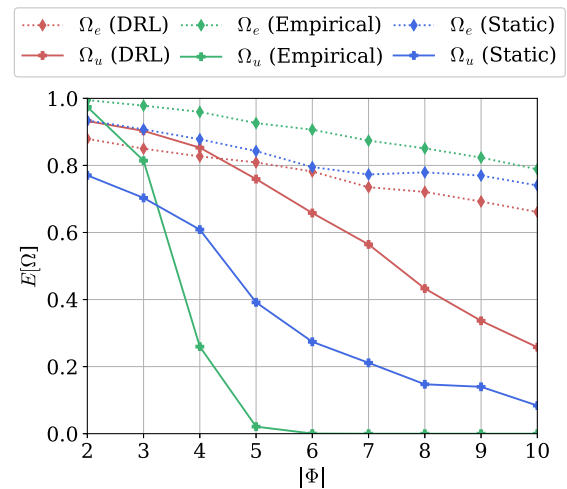


Fig. 11. Expected utility vs flow number.

a specific tuple of learning agents for each network element (as done for the DRL-DP and DRL-TP strategies). As already stated, this stage is performed online and does not require any communication across the network.

In Fig. 11, we plot the expected performance of the eMBB and URLLC slices (i.e., $E[\Omega_e]$ and $E[\Omega_u]$) as a function of the number of information flows in the described scenario. When considering the eMBB flows, all the strategies have a similar behavior since $E[\Omega_e]$ declines gradually as the cardinality of Φ increases. Instead, the performance of the URLLC services is different depending on the chosen approach. Using the empirical algorithm, the URLLC utility is maximized when $|\Phi| = 2$ and abruptly drops to 0 as more flows are initialized over the network. Conversely, both the static and DRL strategies ensures that $E[\Omega_u]$ follows a smoother trend.

In particular, the empirical algorithm outperforms the static approach when the number of information flows is

limited ($|\Phi| \leq 3$). Beyond this point, it becomes more convenient to maintain a static allocation of the system resources to prevent the degradation of the URLLC services. However, both the benchmarks are outperformed by our DRL strategy, which has learned to give priority to the URLLC services, ensuring a better total utility. In particular, using the empirical and the static strategies, $E[\Omega_u]$ falls below 0.3 for $|\Phi| = 6$, while DRL ensures $E[\Omega_u] > 0.6$ in the same conditions.

VII. CONCLUSION

In this work, we investigated the potential of DRL to orchestrate network resources in a NS scenario. Specifically, we developed a distributed DRL system where different units interact to meet the resource demands of multiple information flows. The training of such an architecture was undertaken following an A2C approach, which enables an on-policy training and allows our system to dynamically adapt to different working conditions.

By means of simulations, we showed that the designed strategy can consistently improve the management of network resources, especially when the system complexity, both in terms of network topology and service heterogeneity, increases. In particular, our approach makes it possible to double the number of URLLC flows supported by the network, without degrading the performance of the eMBB flows, and can suit different network topologies without the need for additional training. Besides, transfer learning can be used to further improve the behavior of the learning agents, thus increasing the overall utility at the cost of a reduced adaptability of the agents to different network topologies.

As part of future work, we are interested in extending our NS model by considering more slice classes with different specifications. Particularly, we want to test our framework with real communication traces, with the aim of identifying potential limits. Finally, we will investigate the possibility of introducing additional learning units, which are trained to coordinate the local controllers of our architecture, e.g., by varying the routing paths of the traffic flows.

APPENDIX

In what follows, we derive equation (18), which determines the average queuing time $\tau_{l,\phi}^q(t)$ of ϕ in link l during the timeslot t . Let T^* be the minimum between the slot boundary T and the time at which the queue empties, i.e.,

$$T^* = \min \left(T, \frac{D_{l,\phi}(t-1)}{b_{l,\phi}(t) - b_{l,\phi}^i(t)} \right). \quad (36)$$

Furthermore, let $\mathcal{D}_{l,\phi}(t, u)$ denote the residual backlog of flow ϕ in link l , u seconds after the beginning of timeslot t . Therefore, for any $u \in [0, T^*]$ we have $\mathcal{D}_{l,\phi}(t, u) = D_{l,\phi}(t) - u(b_{l,\phi}(t) - b_{l,\phi}^i(t))$, while $\mathcal{D}_{l,\phi}(t, u) = 0$ for $u \in (T^*, T]$. Now, the queuing delay experienced by the incoming flow at time $u \in [0, T]$ is zero if the queue is empty, and otherwise equal to $\delta(u, t) = \frac{\mathcal{D}_{l,\phi}(u, t)}{b_{l,\phi}(t)}$. The average delay over the timeslot is hence

$$\tau_{l,\phi}^q(t) = \frac{1}{T} \int_0^T \delta(u, t) du = \frac{1}{T} \int_0^{T^*} \frac{\mathcal{D}_{l,\phi}(u, t)}{b_{l,\phi}(t)} du \quad (37)$$

$$= \frac{2T^*D_{l,\phi}(t) - T^{*2}(b_{l,\phi}(t) - b_{l,\phi}^i(t))}{2Tb_{l,\phi}(t)}. \quad (38)$$

For $T^* = T$, we obtain

$$\tau_{l,\phi}^q(t) = \frac{2D_{l,\phi}(t) - T(b_{l,\phi}(t) - b_{l,\phi}^i(t))}{2b_{l,\phi}(t)}. \quad (39)$$

For $T^* < T$, instead, we have $T^*(b_{l,\phi}(t) - b_{l,\phi}^i(t)) = D_{l,\phi}(t)$, so that (38) yields

$$\tau_{l,\phi}^q(t) = \frac{T^*D_{l,\phi}(t)}{2Tb_{l,\phi}(t)} < \frac{D_{l,\phi}(t)}{2b_{l,\phi}(t)}. \quad (40)$$

Recalling that $b_{l,\phi}^o(t)$ used in (18) is defined as the minimum between $b_{l,\phi}(t)$, and $\frac{2D_{l,\phi}(t-1)}{T} + b_{l,\phi}^i(t)$, we can see that (18) is indeed a compact expression for $\tau_{l,\phi}^q(t)$, provided that it is approximated by its upper bound when $T^* < T$.

REFERENCES

- [1] *Service Requirements for Next Generation New Services and Markets*, document Technical Specification 22.261, 3GPP, 2016.
- [2] J. Navarro-Ortiz, P. Romero-Diaz, S. Sendra, P. Ameigeiras, J. J. Ramos-Munoz, and J. M. Lopez-Soler, "A survey on 5G usage scenarios and traffic models," *IEEE Commun. Surveys Tuts.*, vol. 22, no. 2, pp. 905–929, 2nd Quart., 2020.
- [3] M. Yang, Y. Li, D. Jin, L. Zeng, X. Wu, and A. V. Vasilakos, "Software-defined and virtualized future mobile and wireless networks: A survey," *Mobile Netw. Appl.*, vol. 20, no. 1, pp. 4–18, Feb. 2015.
- [4] P. Zhang, H. Yao, and Y. Liu, "Virtual network embedding based on computing, network, and storage resource constraints," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3298–3304, Oct. 2018.
- [5] R. Peter *et al.*, "Network slicing to enable scalability and flexibility in 5G mobile networks," *IEEE Commun. Mag.*, vol. 55, no. 5, pp. 72–79, May 2017.
- [6] I. Afolabi, T. Taleb, K. Samdanis, A. Ksentini, and H. Flinck, "Network slicing and softwareization: A survey on principles, enabling technologies, and solutions," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 3, pp. 2429–2453, 3rd Quart., 2018.
- [7] P. Popovski, K. F. Trillingsgaard, O. Simeone, and G. Durisi, "5G wireless network slicing for eMBB, URLLC, and mMTC: A communication-theoretic view," *IEEE Access*, vol. 6, pp. 55765–55779, 2018.
- [8] M. Richart, J. Baliosian, J. Serrat, and J.-L. Gorricho, "Resource slicing in virtual wireless networks: A survey," *IEEE Trans. Netw. Serv. Manage.*, vol. 13, no. 3, pp. 462–476, Sep. 2016.
- [9] P. Caballero, A. Banchs, G. de Veciana, and X. Costa-Pérez, "Multi-tenant radio access network slicing: Statistical multiplexing of spatial loads," *IEEE/ACM Trans. Netw.*, vol. 25, no. 5, pp. 3044–3058, Dec. 2017.
- [10] K. Samdanis, X. C. Perez, and V. Sciancalepore, "From network sharing to multi-tenancy: The 5G network slice broker," *IEEE Commun. Mag.*, vol. 54, no. 7, pp. 32–39, Jul. 2016.
- [11] R. Trivisonno, R. Guerzoni, I. Vaishnavi, and A. Frimpong, "Network resource management and QoS in SDN-enabled 5G systems," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Dec. 2014, pp. 1–7.
- [12] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [13] H. Halabian, "Distributed resource allocation optimization in 5G virtualized networks," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 3, pp. 627–642, Mar. 2019.
- [14] M. Leconte, G. S. Paschos, P. Mertikopoulos, and U. C. Kozat, "A resource allocation framework for network slicing," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2018, pp. 2177–2185.
- [15] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, Jan. 2011.
- [16] Y. Xiao, M. Hirzallah, and M. Krunz, "Distributed resource allocation for network slicing over licensed and unlicensed bands," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 10, pp. 2260–2274, Oct. 2018.
- [17] M. Hu, L. Zhuang, D. Wu, Y. Zhou, X. Chen, and L. Xiao, "Learning driven computation offloading for asymmetrically informed edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 8, pp. 1802–1815, Aug. 2019.

- [18] F. Fossati, S. Moretti, P. Perny, and S. Secchi, "Multi-resource allocation for network slicing," *IEEE/ACM Trans. Netw.*, vol. 28, no. 3, pp. 1311–1324, Jun. 2020.
- [19] R. Alvizu, S. Troia, G. Maier, and A. Pattavina, "Matureistic with machine-learning-based prediction for software-defined mobile metro-core networks," *J. Opt. Commun. Netw.*, vol. 9, no. 9, pp. 19–30, Sep. 2017.
- [20] Y. Hua, R. Li, Z. Zhao, X. Chen, and H. Zhang, "GAN-powered deep distributional reinforcement learning for resource management in network slicing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 2, pp. 334–349, Feb. 2020.
- [21] A. Thantharate, R. Paropkari, V. Walunj, and C. Beard, "DeepSlice: A deep learning approach towards an efficient and reliable network slicing in 5G networks," in *Proc. IEEE 10th Annu. Ubiquitous Comput., Electron. Mobile Commun. Conf. (UEMCON)*, Oct. 2019, pp. 762–767.
- [22] Y. M. Saputra, D. T. Hoang, D. N. Nguyen, E. Dutkiewicz, D. Niyato, and D. I. Kim, "Distributed deep learning at the edge: A novel proactive and cooperative caching framework for mobile edge networks," *IEEE Wireless Commun. Lett.*, vol. 8, no. 4, pp. 1220–1223, Aug. 2019.
- [23] Y. Hua, R. Li, Z. Zhao, X. Chen, and H. Zhang, "GAN-powered deep distributional reinforcement learning for resource management in network slicing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 2, pp. 334–349, Feb. 2020.
- [24] K. Suh, S. Kim, Y. Ahn, S. Kim, H. Ju, and B. Shim, "Deep reinforcement learning-based network slicing for beyond 5G," *IEEE Access*, vol. 10, pp. 7384–7395, 2022.
- [25] J. Mei, X. Wang, K. Zheng, G. Boudreau, A. B. Sediq, and H. Abou-Zeid, "Intelligent radio access network slicing for service provisioning in 6G: A hierarchical deep reinforcement learning approach," *IEEE Trans. Commun.*, vol. 69, no. 9, pp. 6063–6078, Sep. 2021.
- [26] Y. Kim and H. Lim, "Multi-agent reinforcement learning-based resource management for end-to-end network slicing," *IEEE Access*, vol. 9, pp. 56178–56190, 2021.
- [27] Y. Shao, R. Li, Z. Zhao, and H. Zhang, "Graph attention network-based DRL for network slicing management in dense cellular networks," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Mar. 2021, pp. 1–6.
- [28] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.
- [29] T. Kimura, T. Kimura, A. Matsumoto, and K. Yamagishi, "Balancing quality of experience and traffic volume in adaptive bitrate streaming," *IEEE Access*, vol. 9, pp. 15530–15547, 2021.
- [30] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [31] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ, USA: Wiley, 2014.
- [32] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proc. 13th Annu. ACM Symp. Theory Comput. (STOC)*, 1998, pp. 604–613.
- [33] V. R. Konda and J. N. Tsitsiklis, "Actor-critic algorithms," in *Proc. Adv. Neural Inf. Process. Syst.*, 2000, pp. 1008–1014.
- [34] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
- [35] H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, and L. Daniel, "Efficient neural network robustness certification with general activation functions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 4939–4948.
- [36] T. Jaakkola, S. P. Singh, and M. I. Jordan, "Reinforcement learning algorithm for partially observable Markov decision problems," in *Proc. Adv. Neural Inf. Process. Syst.*, 1995, pp. 345–352.
- [37] X.-S. Yang, *Nature-Inspired Metaheuristic Algorithms*. Frome, U.K.: Luniver Press, 2010.
- [38] D. Whitley, "A genetic algorithm tutorial," *Statist. Comput.*, vol. 4, no. 2, pp. 65–85, Jun. 1994.
- [39] L. Cominardi, L. M. Contreras, C. J. Bernardos, and I. Berberana, "Understanding QoS applicability in 5G transport networks," in *Proc. IEEE Int. Symp. Broadband Multimedia Syst. Broadcast. (BMSB)*, Jun. 2018, pp. 1–5.
- [40] *System architecture for the 5G System (5GS)*, document Technical Specification 23.501, 3GPP, Jan. 2017.
- [41] D. Sattar and A. Matrawy, "DSAF: Dynamic slice allocation framework for 5G core network," 2019, *arXiv:1905.03873*.
- [42] A. Chiha, M. Van der Wee, D. Colle, and S. Verbrugge, "Network slicing cost allocation model," *J. Netw. Syst. Manage.*, vol. 28, no. 3, pp. 627–659, Mar. 2020.
- [43] F. Voigtlander, A. Ramadan, J. Eichinger, C. Lenz, D. Pensky, and A. Knoll, "5G for robotics: Ultra-low latency control of distributed robotic systems," in *Proc. Int. Symp. Comput. Sci. Intell. Controls (ISCSIC)*, Oct. 2017, pp. 69–72.
- [44] J. Sachs *et al.*, "Adaptive 5G low-latency communication for tactile internet services," *Proc. IEEE*, vol. 107, no. 2, pp. 325–349, Sep. 2019.
- [45] X. Jiang *et al.*, "Low-latency networking: Where latency lurks and how to tame it," *Proc. IEEE*, vol. 107, no. 2, pp. 280–306, Feb. 2019.
- [46] *Policy and Charging Control Architecture*, document Technical Specification 23.203, 3GPP, Sep. 2005.
- [47] C. J. Geyer, "Practical Markov chain Monte Carlo," *Stat. Sci.*, vol. 7, no. 4, pp. 473–483, Nov. 1992.
- [48] *GARR High Capacity Network*. Accessed: Mar. 31, 2021. [Online]. Available: <https://www.garr.it/en/infrastructures/network-infrastructure/network-map>



Federico Mason (Graduate Student Member, IEEE) received the bachelor's and master's degrees from the University of Padova, Italy, in 2016 and 2018, respectively, where he is currently pursuing the Ph.D. degree in information engineering. In 2022, he has spent a period as a Visiting Scholar at the Scripps Research Translational Institute, San Diego, CA, USA, where he designed a new technique to analyze electrocardiogram signals. In 2020, he has won the National Grant from the GARR Association to investigate new artificial intelligence algorithms to manage future network slicing systems. His research interests include the development of artificial intelligence solutions to optimize complex engineering systems with a special focus on reinforcement learning and distributed machine learning.



Gianfranco Nencioni received the M.Sc. degree in telecommunication engineering and the Ph.D. degree in information engineering from the University of Pisa, Italy, in 2008 and 2012, respectively. In 2011, he was a Visiting Ph.D. Student with the Computer Laboratory, University of Cambridge, U.K. He was a Post-Doctoral Fellow with the University of Pisa from 2012 to 2015 and the Norwegian University of Science and Technology, Norway, from 2015 to 2018. He has been an Associate Professor with the University of Stavanger, Norway, since 2018. He is currently the Head of the Computer Networks (ComNet) Research Group and the Leader of the 5G-MODaNeI Project funded by the Norwegian Research Council. His research activity regards modeling and optimization in emerging networking technologies, such as SDN, NFV, 5G and beyond, network slicing, and multi-access edge computing. His past research activity has been focused on energy-aware routing and design in both wired and wireless networks and on dependability of SDN and NFV.



Andrea Zanella (Senior Member, IEEE) received the Laurea degree in computer engineering from the University of Padova, Italy, in 1998, and the Ph.D. degree in 2001. In 2000, he was a Visiting Scholar at the University of California, Los Angeles (UCLA), within the Prof. Mario Gerla's Research Team. He is currently a Full Professor at the Department of Information Engineering (DEI), University of Padova. He has authored more than 190 papers. His long-established research activity is in the field of protocol design, optimization, and performance evaluation of wired and wireless networks. He has been serving in the editorial board for several top journals, such as IEEE INTERNET OF THINGS JOURNAL, IEEE COMMUNICATIONS SURVEYS AND TUTORIALS, and IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING.